Ejercicios de Programación orientada a objetos (II)

Programación — DAW

Ricardo Pérez López
IES Doñana

Curso 2025/2026

1. Crear una clase `CuentaBancaria` con:
   - Campos: `titular` y `saldo`.
   - Métodos:
     - `ingresar(`*cantidad*`)`
     - `retirar(`*cantidad*`)`

   No permitir saldo negativo.
2. Modificar la clase `CuentaBancaria` del ejercicio anterior de la siguiente forma:
   - El saldo debe ser privado (`__saldo`).
   - Crear los siguientes métodos:
     - `get_saldo()`
     - `set_saldo(`*valor*`)` (sólo si es positivo)

   Definir el saldo como una propiedad.
3. Diseñar y codificar un modelo orientado a objetos de un banco donde hay cuentas corrientes que tienen un titular y unos movimientos. Los titulares son clientes del banco. Los clientes del banco pueden ser titulares de varias cuentas al mismo tiempo. Los movimientos pertenecen a una sola cuenta. Para ello:

   a) Crear la clase `Cliente` con los atributos `__dni`, `__nombre` y `__apellidos`.

   b) Crear la clase `Movimiento` con los atributos `__concepto` y `__cantidad`. Los movimientos son inmutables.

c) Crear la clase `Cuenta` con los atributos `__numero`, `__titular`, `__movimientos` y `__saldo`.

1) No se puede cambiar el número de una cuenta.

2) Se puede añadir un movimiento a una cuenta, pero no cambiar ni eliminar movimientos.

3) Tampoco se puede modificar directamente el saldo, sino que se debe actualizar directamente a partir de los movimientos que se realicen en la cuenta.

d) Crear un módulo `principal.py` que use las clases anteriores para representar un modelo dinámico de objetos donde el cliente Antonio Martínez tiene dos cuentas corrientes, cada una de ellas con tres movimientos. Imprimir por pantalla el saldo actual de cada cuenta.

e) ¿Cómo se podría implementar la generación automática e incremental del número de cuenta cuando se crea una cuenta nueva? Codificarlo. (*Indicación*: Usar atributos estáticos.)

f) ¿Cómo se podría implementar la colección de cuentas del banco de forma que se pueda acceder de forma eficiente a una cuenta concreta a partir de su número? Codificarlo.

4. Diseñar y codificar un modelo orientado a objetos de una tienda online donde hay clientes, artículos y facturas. Para ello:

a) Crear la clase `Cliente` con los atributos `__dni`, `__nombre` y `__apellidos`.

b) Crear la clase `Articulo` con los atributos `__codigo`, `__denominacion` y `__precio`.

c) Crear la clase `Factura` con los atributos `__numero`, `__cliente` y `__lineas`.

1) Cada línea de factura debe almacenar un artículo y una cantidad.

2) El total de la factura no se debe almacenar, sino que se debe calcular automáticamente sumando el precio de cada artículo multiplicado por la cantidad.

3) Las líneas de una factura pertenecen a esa factura.

4) Las líneas de una factura se pueden añadir o eliminar de una factura, pero no modificar.

d) Crear un módulo `principal.py` que use las clases anteriores para representar un modelo dinámico de objetos donde existe una factura del cliente Rosa González que ha comprado dos televisores de 399 € cada uno y una tarjeta gráfica de 239 €. Imprimir por pantalla todos los datos de la factura como si fuera una factura real, incluyendo el importe total de la misma.

5. Diseñar y codificar un modelo orientado a objetos de una biblioteca donde hay socios que pueden llevarse libros prestados durante quince días como máximo. Para ello:

a) Crear la clase `Lector` con los atributos `__numero`, `__nombre`, `__apellidos` y `__moroso` (éste último es un *booleano* que por defecto estará a `False`).

b) Crear la clase `Libro` con los atributos `__codigo`, `__titulo`, `__autor` y `__editorial`.

c) Crear la clase `Prestamo` con los atributos `__lector`, `__libro`, `__fecha_prestamo` y `__fecha_devolucion`.

1) Los préstamos se crean con `__fecha_devolucion` vacía (es decir, a `None`).

2) Las fechas se pueden almacenar como objetos de la clase `datetime` del módulo `datetime`.

3) Cuando un lector devuelve un libro, se introduce la fecha de devolución en el atributo `__fecha_devolucion` del préstamo correspondiente.

4) El resto de los atributos de un préstamo no se pueden modificar.

5) Si se devuelve un libro pasados más de quince días desde su préstamo, se debe marcar al lector como moroso.

6) Se debe impedir que un libro ya prestado se pueda volver a prestar. Para ello hay dos técnicas:

a′ Crear y usar un atributo *booleano* `__prestado` en la clase `Libro`, de forma que se ponga a `True` cuando se preste y a `False` cuando se devuelva.

b′ Comprobar en los objetos `Prestamo` si ese libro está prestado y aún no se ha devuelto.

d) Crear un módulo `principal.py` que use las clases anteriores para representar un modelo dinámico de objetos donde haya dos lectores (Ana García y Roberto Sánchez) y tres libros («*El camino*» de Miguel Delibes, «*Cien años de soledad*» de Gabriel García Márquez y «*Rayuela*» de Julio Cortázar).

e) Prestar dos libros a un lector y el otro al otro lector.

f) Intentar prestar un libro que ya está prestado.

g) Intentar devolver un libro prestado con más de quince días de préstamo.

6. Traducir a clases y objetos todo el código que tenemos actualmente en `https://github.com/iesdonana/vampiro.git`, teniendo en cuenta que:

a) Las localidades deben ser objetos de la clase `Localidad`.

b) Es posible que no sea necesario hacer una clase `Localidades`.

c) El jugador debe ser un único objeto de la clase `Jugador` (a ésto se le llama *Singleton*).

d) Las conexiones de una localidad se deben almacenar dentro de la localidad.

e) Cada conexión puede ser un objeto de una clase `Conexion`, o puede que no merezca la pena crear una clase para eso. En tal caso, las conexiones serían parejas de elementos (que se podrían representar con cualquier estructura tipo lista, tupla, diccionario…) que contenga una dirección y una localidad de destino.

f) Los ítems (objetos que aparecen en el juego, como el crucifijo o la ristra de ajos) deben ser objetos de la clase `Item`.

g) Las colecciones deben ser objetos de la clase `Coleccion`.

h) Los ítems pueden estar en una localidad o en el inventario del jugador. Para ello, hay que usar objetos de la clase `Coleccion`.

i) Un *token* representa una palabra con significado propio y distinto de otras palabras. Cada *token* debe ser un objeto de la clase `Token`.

j) Cada grupo de palabras del mismo tipo («verbo», «nombre», etc.) debe ser un objeto de la clase `Vocabulario`. Así, debe haber un vocabulario que contenga todos los verbos, otro que contenga todos los nombres, etc.

Cada objeto de la clase `Vocabulario` debe inicializarse con un diccionario que asocie cada *token* con una lista de lexemas que sean sinónimos.

Por ejemplo:

```
ABRIR = Token()
ARRIBA = Token()
verbos = Vocabulario({
    ABRIR: ['ABRIR', 'ABRE'],
    ARRIBA: ['ARRIBA', 'SUBIR', 'SUBE']
})
CRUCIFIJO = Token()
AJOS = Token()
nombres = Vocabulario({
    CRUCIFIJO: ['CRUCIFIJO', 'CRUZ'],
    AJOS: ['AJOS', 'AJO', 'RISTRA']
})
```

k) La función `interpretar` debe implementarse como un método estático de una clase (que puede ser la misma clase `Vocabulario`) que se encargue de analizar sintácticamente la entrada del jugador a partir de los vocabulario de verbos y de nombres. Ese método debe devolver los *tokens* del verbo y el nombre encontrados, o un *token* especial (llamado *token nulo*) que represente que no se ha encontrado el verbo o el nombre.

7. Crear la clase `Persona` con un método `compara_edad` que compare la edad de una persona con la de otra.

   Ejemplos:

```
>>> p1 = Persona('Samuel', 24)
>>> p2 = Persona('Jael', 36)
>>> p3 = Persona('Liliana', 24)
>>> p1.compara_edad(p2)
'Jael es más viejo que yo.'
>>> p2.compara_edad(p1)
'Samuel es más joven que yo.'
>>> p1.compara_edad(p3)
'Liliana tiene la misma edad que yo.'
```

Fuente:

`https://edabit.com/challenge/JFLADuABfkeoz8mqN`

8. Crear la clase `Empleado` con atributos `nombre` y `apellidos`. A partir de ellos, crear los atributos `nombre_completo` y `email` de la siguiente forma:

   - El `nombre_completo` será simplemente el nombre y los apellidos unidos con un espacio en blanco.
   - El `email` se forma con el nombre y los apellidos (todo en minúsculas) unidos con un `.` intermedio y seguido de `@company.com`.

   Ejemplos:

```
>>> emp1 = Empleado('Juan', 'Pérez')
>>> emp2 = Empleado('María',  'García')
>>> emp3 = Empleado('Antonio', 'González')
>>> emp1.nombre_completo
'Juan Pérez'
>>> emp2.email
'maría.garcía@company.com'
>>> emp3.nombre
'Antonio'
```

   Fuente:

   `https://edabit.com/challenge/gB7nt6WzZy8TymCah`

9. Las instancias de la clase `Empleado` tienen los atributos `nombre`, `apellidos` y `salario`. Crear, además, el método estático `desde_cadena` que analiza una cadena que contiene esos tres valores separados por un guion y los asigna a sus atributos correspondientes.

   Ejemplos:

```
>>> emp1 = Empleados('María', 'García', 60000)
>>> emp2 = Empleados.desde_cadena('Juan-Pérez-55000')
>>> emp1.nombre
'María'
>>> emp1.salario
60000
>>> emp2.nombre
'Juan'
>>> emp2.salario
55000
```

Fuente:

https://edabit.com/challenge/j2HauiSdDadkjxjsQ

10. Escribir un programa que lea por la entrada una hora cualquiera y un número *n* que representa una cantidad en segundos. El programa mostrará la hora introducida y las *n* siguientes, que se diferencian en un segundo. Para ello, hemos de diseñar previamente la clase `Hora` que dispone de los campos `hora`, `minuto` y `segundo`. Los valores de los campos se controlarán mediantes métodos *get*/*set*.

11. Diseñar la clase `Texto` que gestiona una cadena de caracteres con algunas características:

    - La cadena de caracteres tendrá una longitud máxima que se especifica en el constructor.
    - Permite añadir un carácter al principio o al final, siempre y cuando no se exceda la longitud máxima (es decir, hay espacio disponible).
    - Igualmente, permite añadir una cadena, al principio o al final del texto (siempre y cuando no se rebase el tamaño máximo establecido).
    - Cada objeto de tipo `Texto` tiene que conocer la fecha en la que se creó, así como la fecha y hora de la última modificación efectuada.
    - Deberá existir un método que imprima por la salida la información que gestiona cada texto (el texto en sí, la fecha de creación y la fecha y hora de la última modificación).

12. Definir una clase que permita controlar un sintonizador digital de emisoras FM. Concretamente, se desea dotar al controlador de una interfaz que permita subir o bajar la frecuencia (en saltos de 0,5 MHz) y mostrar la frecuencia sintonizada en un momento dado. Supondremos que el rango de frecuencias para manejar oscila entre los 80 MHz y los 108 MHz y que, al inicio, el controlador sintonice la frecuencia indicada en el constructor u 80 MHz por defecto. Si durante una operación de subida o bajada se sobrepasa uno de los dos límites, la frecuencia sintonizada debe pasar a ser la del extremo contrario. Escribir un pequeño programa principal para controlar su funcionamiento.

13. Modelar una casa con muchas bombillas, de forma que cada bombilla se puede encender o apagar individualmente. Para ello, hacer una clase `Bombilla` con una variable privada que indique si está encendida o apagada, así como un método que nos diga el estado de una bombilla concreta. Además, queremos poner un interruptor general, de forma que si este se apaga, todas las bombillas quedan apagadas, según estuvieran antes. Cada bombilla se enciende y se apaga individualmente, pero sólo responde que está encendida si su interruptor particular está activado y además hay luz general.

14. Hemos recibido el encargo de un cliente para definir las clases necesarias (sólo implementar los campos y los constructores) para gestionar una empresa ferroviaria, en la que se distinguen dos grandes grupos: el personal y la maquinaria. En el primero se ubican los empleados de la empresa, que se clasifican en tres grupos: los maquinistas, los mecánicos y los jefes de estación. De cada uno de ellos es necesario guardar:

    - Maquinistas: su nombre, DNI, sueldo y el rango que tienen adquirido.
    - Mecánicos: su nombre, teléfono y en qué especialidad desarrollan su trabajo (esta puede ser: frenos, hidráulica, electricidad o motor).
    - Jefes de estación: su nombre, DNI y la fecha en la que fue nombrado jefe de estación.

    En la parte de maquinaria podemos encontrar trenes, locomotoras y vagones. De cada uno de ellos hay que considerar:

    - Vagones: tienen un número que los identifica, una carga máxima (en kilos), la carga actual y el tipo de mercanciía con la que están cargados.
    - Locomotoras: disponen de una matrícula (que las identifica), la potencia de sus motores y una antigüedad (año de fabricación). Además, cada locomotora tiene asignado un mecánico que se encarga de su mantenimiento.
    - Trenes: están formados por una locomotora y un máximo de 5 vagones. Cada tren tiene asignado un maquinista que es responsable de él.

Soluciones

1. *a)*
```
class Cliente:
    def __init__(self, dni, nombre, apellidos):
        self.__dni = dni
        self.__nombre = nombre
        self.__apellidos = apellidos

    def dni(self):
        return self.__dni
```

```
    def nombre(self):
        return self.__nombre

    def apellidos(self):
        return self.__apellidos
```

b)
```
class Movimiento:
    def __init__(self, concepto, cantidad):
        self.__concepto = concepto
        self.__cantidad = cantidad

    def concepto(self):
        return self.__concepto

    def cantidad(self):
        return self.__cantidad
```

c)
```
class Cuenta:
    def __init__(self, numero, titular):
        self.__numero = numero
        self.__titular = titular
        self.__movimientos = []
        self.__saldo = 0

    def numero(self):
        return self.__numero

    def titular(self):
        return self.__titular

    def saldo(self):
        return self.__saldo

    def nuevo_movimiento(self, concepto, cantidad):
        self.__movimientos.append(Movimiento(concepto, cantidad))
        self.__saldo += cantidad
```

d)
```
antonio = Cliente('38475923M', 'Antonio', 'Martínez')
c1 = Cuenta(1, antonio)
c1.nuevo_movimiento('Ingreso', 100)
c1.nuevo_movimiento('Retirada', -30)
c1.nuevo_movimiento('Ingreso', 20)
c2 = Cuenta(2, antonio)
c2.nuevo_movimiento('Ingreso', 500)
```

```
c2.nuevo_movimiento('Ingreso', 80)
c2.nuevo_movimiento('Retirada', -40)
print(c1.saldo())
print(c2.saldo())
```

e) En la clase `Cuenta` guardamos un diccionario como una variable de clase, cuya clave sería el número de la cuenta y cuyo valor sería la cuenta correspondiente a ese número. Además, creamos un método estático que devuelva la cuenta a partir de su número:

```
class Cuenta:
    __cuentas = {}

    def __init__(self, numero, titular):
        self.__numero = numero
        self.__titular = titular
        self.__movimientos = []
        self.__saldo = 0
        Cuenta.__cuentas[numero] = self

    @staticmethod
    def buscar(numero):
        return Cuenta.__cuentas.get(numero)
```

Y luego se podría buscar una cuenta así:

```
c = Cuenta.buscar(2)
```

2. *a)*

```
class Cliente:
    def __init__(self, dni, nombre, apellidos):
        self.__dni = dni
        self.__nombre = nombre
        self.__apellidos = apellidos

    def dni(self):
        return self.__dni

    def nombre_completo(self):
        return self.__nombre + ' ' + self.__apellidos
```

b)

```
class Articulo:
    def __init__(self, codigo, denominacion, precio):
        self.__codigo = codigo
        self.__denominacion = denominacion
        self.__precio = precio
```

```
    def codigo(self):
        return self.__codigo

    def denominacion(self):
        return self.__denominacion

    def precio(self):
        return self.__precio
```

c)
```
class Factura:
    def __init__(self, numero, cliente, lineas):
        self.__numero = numero
        self.__cliente = cliente
        self.__lineas = lineas.copy()

    def numero(self):
        return self.__numero

    def cliente(self):
        return self.__cliente

    def total(self):
        tot = 0
        for art, cantidad in self.__lineas.items():
            tot += cantidad * art.precio()
        return tot

    def imprimir(self):
        print(f'Factura n.º {self.numero()}:')
        print()
        cli = self.cliente()
        print(f'Cliente: {cli.dni()} {cli.nombre_completo()}')
        print()
        print('Cód. Denominación         Cantidad    Precio     Importe')
        print('-' * 56)
        total = 0
        for art, cantidad in self.__lineas.items():
            imp = cantidad * art.precio()
            total += imp
            print(f'{art.codigo():>4} ', end='')
            print(f'{art.denominacion():<20} ', end='')
            print(f'{cantidad:>8} ', end='')
            print(f'{art.precio():>9} ', end='')
            print(f'{imp:>11}')
```

```
        print('-' * 56)
        print(f'TOTAL: {total:>49}')
```

d)
```
rosa = Cliente('12312312A', 'Rosa', 'González')
tv = Articulo(1, 'Televisor', 399)
grafica = Articulo(2, 'Tarjeta gráfica', 239)
f = Factura(1, rosa, {tv: 2, grafica: 1})
f.imprimir()
```

Produce el siguiente resultado:

```
Factura n.º 1:

Cliente: 12312312A Rosa González

Cód. Denominación         Cantidad    Precio     Importe
--------------------------------------------------------
   1 Televisor                   2       399         798
   2 Tarjeta gráfica             1       239         239
--------------------------------------------------------
TOTAL:                                                1037
```